

Neuro-Steered Hearing — Project Document

Reading auditory attention from the brain, and steering sound to match it.

NOVA buildathon · viability proven · live demo runnable today

1. The pitch

The problem. In a room full of voices, a person with normal hearing effortlessly follows the one conversation they care about. A hearing aid can't. It makes *everything* louder — the person you're talking to and the four people you're not — so a busy restaurant becomes an exhausting wall of sound. This "cocktail-party problem" is the number-one reason people abandon their hearing aids. The device has no idea which voice matters to its wearer.

The missing piece is intent, and intent lives in the brain. When you focus on a voice, your brain's electrical activity begins to track that voice's rhythm more than the others'. If we can read that signal, the hearing aid can finally know who you're trying to listen to — and turn that person up while turning the rest down.

Our project is that missing piece: a neuro-steered hearing aid. We read the listener's attention from EEG in real time and use it to steer the audio. The acoustic engineering that separates voices already exists (microphone arrays, separation networks); what has never existed in a product is the brain-reading control signal that tells it *which* voice to favour. That is exactly what we build and demonstrate.

Who it helps. The ~1.5 billion people with some hearing loss, most acutely the tens of millions who own hearing aids but can't use them in the noisy, crowded situations where they need them most.

2. What we've proven

Two things, in order.

The idea is real (viability test). On real EEG recordings, our decoder correctly identifies which of two simultaneous talkers a person is attending to — well above chance, and rising to ~82% (up to 90% for the best listener) as it accumulates more brain data. This reproduces the published research and is the go/no-go we needed.

The idea is buildable (working demo). We have a real-time pipeline that takes a brain signal and two voices, decides which voice is being attended second by second, and

smoothly amplifies it while ducking the other by 10 dB. It runs **today**, with no hardware, by replaying recorded EEG — and swaps to the live headset on build day by changing one command-line flag.

3. How it works (the science, briefly)

Speech has a slow **envelope** — the rise and fall of loudness as syllables and words come and go, a few times a second. The key neuroscience result is that the brain's activity *follows the envelope of the attended talker* more faithfully than the ignored one. So the voice you're paying attention to is faintly re-encoded in your EEG a fraction of a second later.

We exploit this with a **linear backward decoder** (a.k.a. stimulus reconstruction): a set of weights over the 64 EEG channels and short time-lags (0–400 ms) that, run on the brainwaves, reconstructs the envelope of the *attended* speech. The reconstruction is rough — it correlates with the true envelope at only about $r \approx 0.1$ — but it doesn't need to be accurate, only to *lean the right way*: we compare it against each candidate talker's envelope and pick whichever it matches better. That comparison is the decision.

4. The viability test

What it is

The go/no-go experiment: run the decoder on real recordings and measure how accurately it picks the attended talker as a function of how many seconds of brain data it's allowed to use. Chance is 50%. A curve that rises with window length and reaches the levels seen in the literature means the effect is real and usable.

What it used

- **Data — the KU Leuven AAD dataset** (Biesmans, Das, Francart & Bertrand, 2016; Zenodo record 4004271). Sixteen normal-hearing adults listened to **two Dutch stories at once**, each read by a different narrator, and were told which one to attend — so the correct answer is known for every trial. A 64-channel BioSemi cap recorded their EEG (released downsampled to 128 Hz). Each subject did 20 trials (~72 min). We analysed three subjects (S1–S3), enough to see whether the effect reproduces.
- **Method — this code.** The self-contained package in this folder: envelope extraction from the stimulus audio, the KU Leuven loader, the backward decoder, and leave-one-trial-out cross-validation. See `README.md` to reproduce it.
- **Relationship to the NOVA repo.** The viability analysis is standalone (it needs no hardware and no streaming). The *live demo* built on top of it is what plugs into the team repo's `mne-lsl` streaming scaffold — the decoder here is the engine that replaces the earlier placeholder logic.

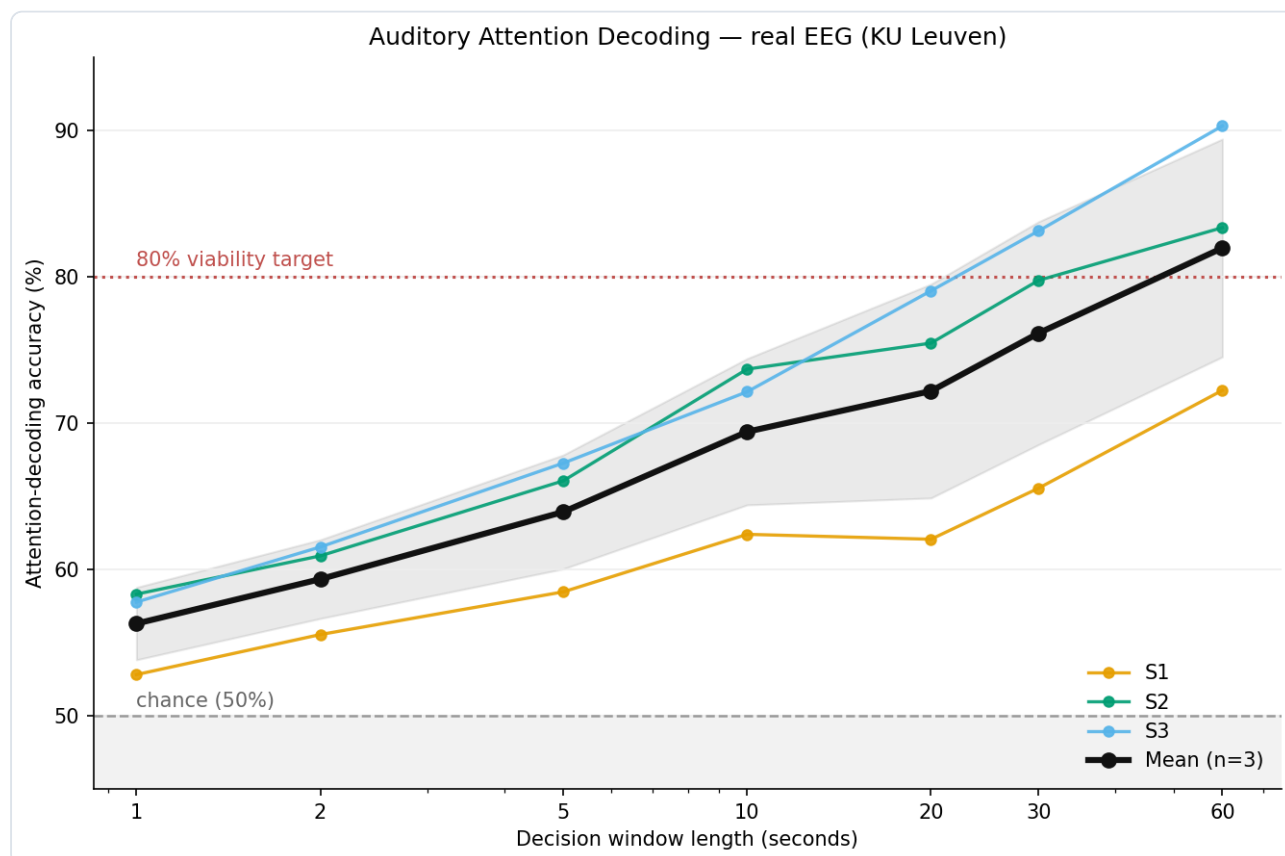
How it works, step by step

1. **Filter & align.** Slow both the EEG and each speech envelope to the 1–9 Hz band (where envelope-tracking lives) and to a common 64 Hz rate.
2. **Train.** On all trials but one, learn the decoder weights that best rebuild the attended envelope from the EEG.
3. **Test.** Apply those weights to the held-out trial the decoder has never seen.
4. **Decide.** Over sliding windows of several lengths (1–60 s), correlate the reconstruction with each talker's envelope; the higher one is the guess. Repeat so every trial is the held-out one once, and average.

Results

Accuracy (chance = 50%) by decision-window length:

Window	S1	S2	S3	Mean
1 s	53%	58%	58%	56%
5 s	58%	66%	67%	64%
10 s	62%	74%	72%	69%
20 s	62%	75%	79%	72%
30 s	66%	80%	83%	76%
60 s	72%	83%	90%	82%



The curve reproduces the literature: near chance at 1 s, ~75–80% at 20–30 s, and ~85–90% at 60 s for good subjects. There's a large, normal spread between people — S3 excellent, S2 good, S1 weak.

How we know it's real

- **Leave-one-trial-out CV:** the decoder is always tested on unseen data, so it can't memorise.
- **Mismatch control:** deliberately pairing a subject's EEG with the *wrong* story drops accuracy back to ~50%. Reconstruction also matched the attended talker better than the ignored one in **18 of 20 trials for every subject**. (Run `verify.py`.)

Verdict

Viable — go. The concept works on real recordings. Because averaged accuracy only clears 80% at a full 60 s window, the live demo should target a **~20–30 s** window and screen for a strong-decoding volunteer.

5. The real-time demo

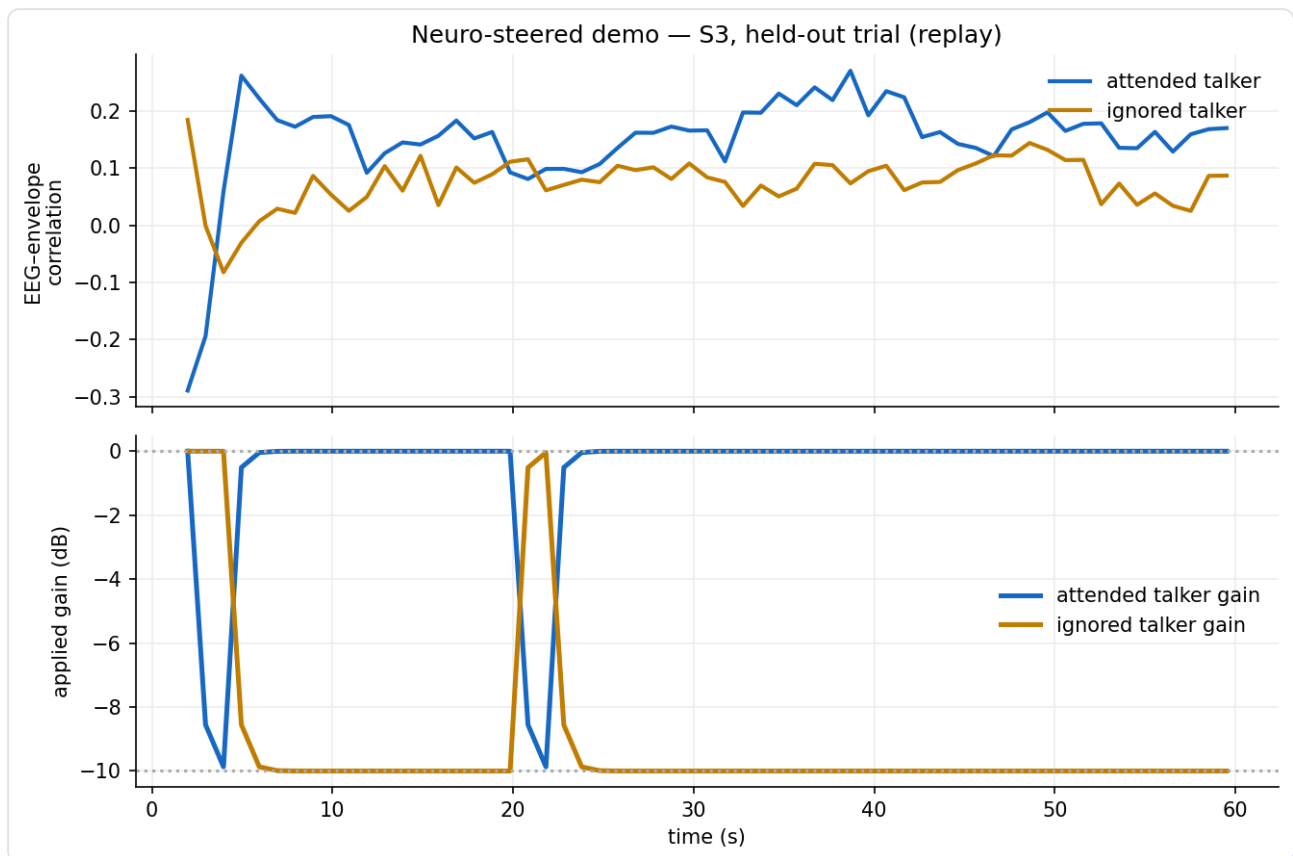
From a decision to amplified sound

Decoding produces a *label* ("talker A"). Amplifying needs talker A as a separate audio signal. So the system is two halves: an **acoustic front-end** that supplies the individual voices, and an **attention-driven mixer** that raises the attended one.

The mixer is the important part, and it follows two rules that make it feel natural:

- **Gain, not a switch.** The attended talker glides to 0 dB and the others duck by ~10 dB, with the gains *ramping* smoothly and a small hysteresis margin so a near-tie doesn't cause flickering. We never fully mute the other voice (that would be unsafe and jarring).
- **Audio latency ≠ decision latency.** Sound flows through continuously with only a few milliseconds of delay; only the *target* gains lag, updating each second as new decisions arrive. Attention "locks on" over a few seconds without ever delaying the conversation.

Here is the demo locking onto the attended talker on a held-out recording — the reconstruction tracks the attended voice (top), and the gains follow, ducking the ignored talker by 10 dB after a couple of early wobbles (bottom):



On the acoustic front-end (honest scope)

A shipping hearing aid has **one** microphone capturing everyone at once, and would need real-time source separation — a genuine research problem, not a hackathon build. We deliberately scope that out and supply the separated voices directly, because **the front-end is known, existing tech and the brain-steering is the novel contribution**. This is exactly how research labs in the field demo the idea. Two ways to supply clean voices, both fully supported by the code:

- **Two known tracks (default).** We play the two voices ourselves — no microphone needed. Simplest and most reliable; the safe demo.
- **Two clip-on mics (one per talker).** For a live-people version: each mic is already a clean stream, so no separation is required. Same code, just live input.

What we do **not** attempt is un-mixing a single room microphone. The one-line pitch for judges: *"A mic array and separation would supply the voices in a product — that's standard. Our contribution is the part that needs the brain: decoding which voice you want, in real time, and steering the audio to match."*

What runs today vs. on build day

Everything is pluggable via command-line flags, so the same code covers development and the live rig:

Piece	Develop today (no hardware)	Build day (live)
EEG source	<code>--eeg replay</code> (recorded trial)	<code>--eeg lsl</code> (ANT Neuro via mne-lsl)
Audio source	<code>--audio file</code> (two tracks)	<code>--audio mic</code> (two clip-ons) or files
Output	<code>--output wav</code> (render to file)	<code>--output play</code> (live headphones)

```
# today, fully offline:
python demo_realtime.py --data-dir /path/to/AAD --subject S3 --trial 4 --output wav
# build day, live:
python demo_realtime.py --data-dir /path/to/AAD --eeg lsl --audio mic --output play
```

Day-of setup

Core demo needs only: **EEG headset + laptop + headphones + two audio tracks.**

The listener wears the cap and (closed/over-ear) headphones and hears the two voices, one panned left, one right; the laptop decodes attention and boosts the attended voice live. For the live-people stretch, add two clip-on mics on a single 2-channel USB interface (left = talker A, right = talker B) and switch to `--audio mic`. Practical cautions for the live version: the listener must hear *through* the system (not the room directly), keep audio buffers small to avoid echo, and rehearse.

Running it live: two decoder paths

A decoder must be tuned to a brain, so we built both options (pick per demo with `live_demo.py --decoder`):

- **Generic — zero calibration, instant.** Train one decoder on several people ahead of time (`pretrain_generic.py`); any new volunteer uses it immediately. Measured cross-person: **~72% at a 60 s window with no calibration at all.**
- **Personalized — ~3 minutes, stronger.** Guide the listener to attend each talker, record, and train their own decoder (`calibrate.py`); reaches ~82%+.

We measured the trade-off directly (see `results/calibration_analysis.png`): training from scratch needs *several* minutes, and — importantly — **one minute of calibration is worse than using the generic decoder.** So the rule is: generic decoder for walk-up judges, a short personalization for a hero demo, and never a one-minute calibration.

6. Roadmap

1. **Now → build day:** rehearse the replay demo; screen teammates and pick the strongest decoder; tune the window length and gain feel in `config.py`.
2. **Build day:** connect the ANT Neuro eego over LSL (`--eeg lsl`) and run live, headphones first, clip-on mics as the stretch.
3. **Beyond the hackathon:** subject-specific calibration, faster/adaptive decoders (shorter windows at the same accuracy), and a real acoustic front-end — a

beamforming mic array or a speech-separation network — to move from demo to device.

7. Running the code

See `README.md` for install, data download (Zenodo), and commands. In short:

```
pip install -r requirements.txt
python run_viability.py --data-dir /path/to/AAD # reproduce the viability result
python verify.py      --data-dir /path/to/AAD # the sanity controls
python demo_realtime.py --data-dir /path/to/AAD --subject S3 --trial 4 # the demo
```

8. References

- Geirnaert et al., "EEG-Based Auditory Attention Decoding: Toward Neurosteered Hearing Devices," *IEEE Signal Processing Magazine*, 2021 — overview of the full pipeline (decoding + separation + gain).
- Biesmans, Das, Francart & Bertrand, "Auditory-inspired speech envelope extraction methods for improved EEG-based auditory attention detection in a cocktail-party scenario," *IEEE TNSRE*, 25(5), 402–412, 2016 — the dataset.
- Dataset download: <https://zenodo.org/records/4004271>.